



招聘流程 AI workflow优化方案 V3.0

脱敏案例 | 企业名称与业务数据已匿名化处理



目录

一、执行摘要 / 提案亮点.....2

二、V3.0 工作流图.....3

三、核心 AI 工具定义（Prompt 设计部分见附件）4

四、评估与分析.....7

五、下一步行动.....14

附件：15

将招聘周期从 60 天压缩至 48 天，JD 起草效率提升 75%以上——这不是愿景，而是 V3.0 智能工作流的交付承诺。

某招聘科技企业的招聘痛点，我们比您更清楚：非结构化的邮件需求、错配的技能描述、反复返工的 JD 审核……每一轮都在吞噬业务增长的时间。为此，我们设计了 V3.0 全流程 AI 辅助招聘工作流——在不替代人类决策的前提下，将 AI 的速度与人类的判断完美融合。

- ✔ 前置一致性校验：自动对比邮件与电话笔记，识别互斥矛盾，HITL 节点精准澄清，从源头杜绝错误传递。
- ✔ 技能栈动态图谱：实时对齐公司真实技术栈，自动拦截“过时/错配”技能要求，告别 Java 8 vs Java 17 的尴尬。
- ✔ 岗位智能分类定级：技术、销售、通用岗分路径处理，初级/中高级/管理层内部适配，生成差异化 JD。
- ✔ 销售岗质量闭环：EVP 评分→AI 微调→人工审核→熔断保护，迭代≤3 次即输出高融合度 JD。

价值直击：JD 起草从 2-3 小时→30 分钟，审阅周期从 3-4 天→半天，月度 ROI 估算达 92.7 倍。4 周技术岗试点即可验证效果，零风险起步。

让招聘回归业务本身，而不是在文档中内耗。

一、执行摘要 / 提案亮点

引子/痛点

招聘经理非结构化需求导致 JD 起草周期长、返工多。

解决方案亮点

通过 AI 一致性校验、岗位分类定级、技能栈动态图谱校验与质量迭代循环，实现 JD 快速生成与高质量定稿。

核心价值/KPIs

JD 起草耗时从 2-3 小时缩短至 30 分钟，审阅周期从 3-4 天降至半天。

关键考量/风险

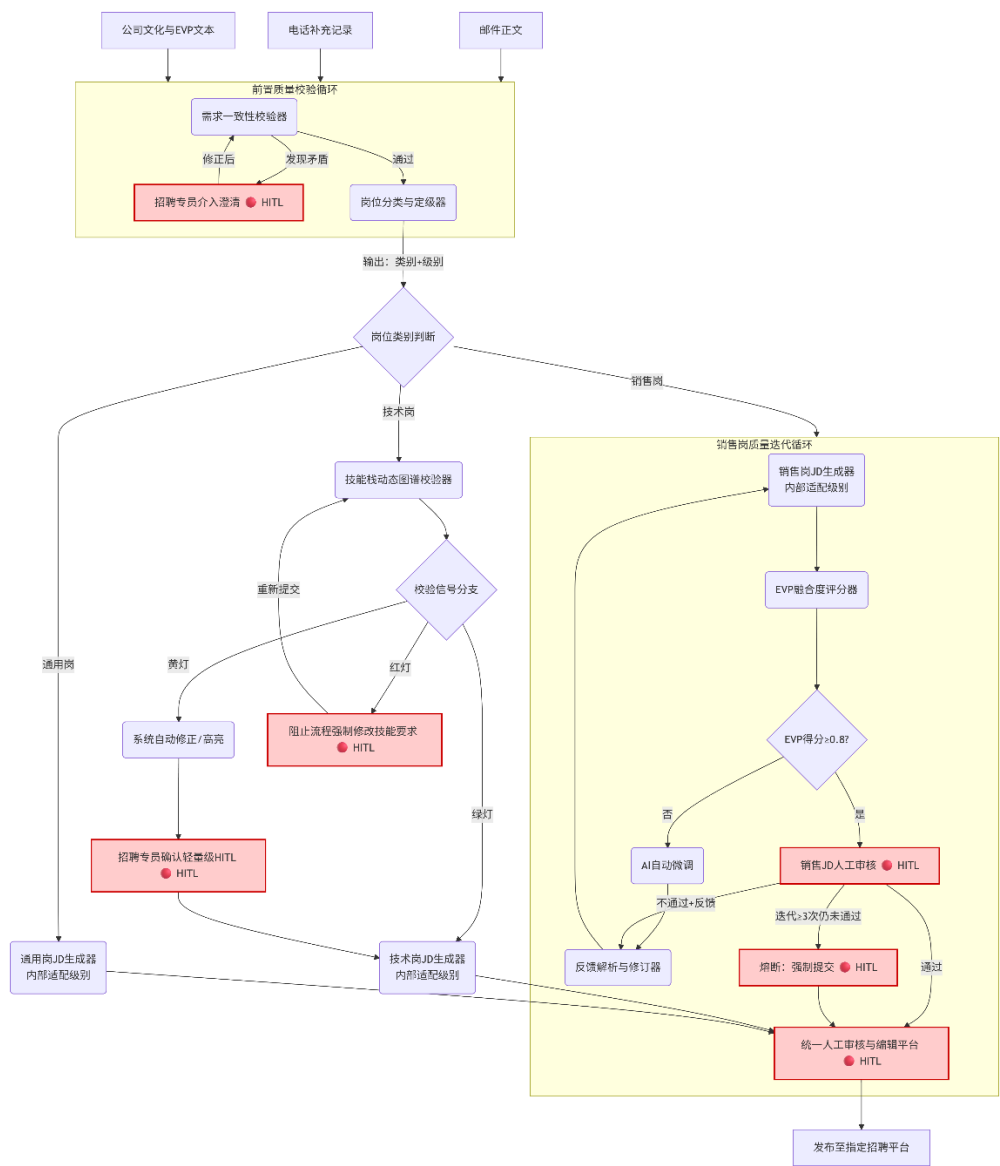
AI 判断可能存在误差，但通过 HITL 节点（人工确认矛盾、审核 JD、技能栈校验干预）可有效兜底。

下一步建议

建议在技术岗招聘中开展为期 4 周的小范围试点，验证 JD 生成效率与质量提升效果。

二、V3.0 流程图

下图展示了完整的高级工作流，包含前置质量校验循环、岗位分类与定级、技术岗技能栈动态图谱校验器、销售岗质量迭代循环以及通用岗流程。



图注：红色背景矩形为必须人工介入的 HITL 节点；菱形为条件分支；圆角矩形为自动化工具或生成器（内部已适配级别）。

三、核心 AI 工具定义（Prompt 设计部分见附件）

工具 1：需求一致性校验器 v1.0

目的

在招聘流程开始时，自动对比招聘经理的邮件需求与 HR 的电话补充笔记，识别完全互斥的矛盾，输出结构化一致性报告。若发现矛盾，触发 HITL（招聘专员介入澄清）。

概念性输入

- 招聘经理需求邮件文本
假设关键字段：职位名称、职责描述片段、技能关键词、经验要求、项目优先级
- HR 电话沟通补充笔记
假设关键字段：职位名称、职责描述、技能要求、经验要求、项目紧急程度
- 公司文化与 EVP 文本（保留，本工具不直接使用）

核心 Prompt 设计思路

以角色扮演（HR 需求一致性分析专家）为基础，逐字段对比 5 个字段（职位名称、经验年限、核心职责、技能要求、项目优先级）。仅当两者说法“完全互斥”时才记为矛盾；补充或细化不算矛盾。输出 JSON 包含 is_consistent 和 conflicts 数组（含字段名和自然语言描述）。

【示例输出】

无矛盾时：{"is_consistent": true, "conflicts": []}

有矛盾时：{"is_consistent": false, "conflicts": [{"field": "experience_years", "description": "邮件要求 3-5 年，电话要求 5-8 年，互斥"}]}

概念性输出

JSON 对象，包含 is_consistent（布尔）和 conflicts（数组）。

工具 2：岗位分类与定级器 v1.0

目的

根据邮件、电话笔记及一致性校验后的信息，自动判断岗位类别（技术岗/销售岗/通用岗）和级别（初级/中高级/管理层），驱动后续分支路径和内部适配。

概念性输入

- 邮件正文、电话补充记录
- 一致性校验器输出的结构化 JSON（如有矛盾已解决）
假设关键字段：职位名称、职责描述片段、经验年限、关键词（如“带领团

从")

核心 Prompt 设计思路

- 类别判断：基于职位名称和职责关键词（技术/销售/通用）。
- 级别判断：依据经验年限和职责复杂度（0-2 年→初级，3-5 年→中高级，6 年+或带团队→管理层）。优先更高层级，无明确信息默认“中高级”。

输出 {"job_category": "...", "job_level": "..."}。

【示例输出】

{"job_category": "技术岗", "job_level": "中高级"}

概念性输出

包含 job_category 和 job_level 的 JSON 对象。

工具 3：EVP 融合度评分器 v1.0

目的

评估销售岗 JD 草稿与公司 EVP 要素的匹配程度，驱动质量迭代循环（得分 ≥ 0.8 进入人工审核，否则触发 AI 微调）。

概念性输入

- 销售岗 JD 草稿（完整四栏）
- 公司 EVP 文本

核心 Prompt 设计思路

要求 LLM 根据 EVP 文本判断 JD 草稿对各 EVP 要素的覆盖程度，输出 0-1 之间的浮点数分数和简短评语。评分标准：0.8 以上为“良好”。

【示例输出】

{"score": 0.85, "feedback": "EVP 中'职业发展路径'和'前沿技术项目'融入较好，但'多元与包容'未体现，建议补充"}

概念性输出

JSON 对象，包含 score（浮点数）和 feedback（字符串）。

工具 4：反馈解析与修订器 v1.0

目的

将人工审核的反馈意见（自然语言）解析为具体修订指令，并生成修订后的 JD 草稿（修改部分加粗高亮）。

概念性输入

- 当前 JD 草稿
- 人工反馈文本（如：“职责部分缺少数据可视化要求，任职要求中 Python 应放在首位”）

核心 Prompt 设计思路

理解反馈意图，在保持原稿结构的前提下对指定部分修改，修改处使用 **加粗** 标记，输出完整修订稿。

【示例输出】

(完整 JD 文本，其中修改部分如“负责数据可视化看板的设计与实现”以加粗显示)

概念性输出

完整 JD 草稿 (Markdown 格式)，修改部分已加粗。

工具 5：熔断机制控制器 v1.0

目的

当销售岗质量迭代循环达到最大次数 (≥ 3 次) 仍未通过人工审核时，触发强制提交并向上级发送升级提醒。

概念性输入

- 当前迭代次数 (整数)
- 最后一次修订的 JD 草稿
- 未通过的反馈记录
- 职位名称

逻辑规则

若迭代次数 ≥ 3 且仍不通过，则：① 强制将当前 JD 发送至分支汇合点；② 生成升级提醒消息，发送给 HRBP 和招聘总监。

【示例输出】

升级提醒消息：“【熔断提醒】数据科学家岗位 JD 经过 3 次迭代仍未通过人工审核，已强制提交。请 HRBP 关注。未通过原因摘要：EVP 融合度不足，职责描述与业务需求有偏差。”

概念性输出

强制提交信号 + 升级提醒消息 (文本)。

工具 6：技能栈动态图谱校验器 v1.0

目的

在技术岗 JD 生成之前，将岗位技能需求与公司动态技术栈图谱对比，识别技能过时、错配、强制缺失等问题，输出绿/黄/红灯信号并触发相应动作。

概念性输入

- 动态技术栈图谱 (JSON)：包含 core_stack (核心技术)、legacy_stack (淘汰中技术+替代项)、future_stack (未来技术)
- 岗位技能需求文本 (从分类器输出及原始需求提取)
- 职位名称

核心 Prompt 设计思路

按优先级执行：

1. 强制缺失检测：若缺失 core_stack 任一技术 $\rightarrow$ 红灯
2. 冲突与过时检测：若出现 legacy_stack 且含强制词 $\rightarrow$ 黄灯；若

与 core_stack 严重互斥 → 红灯

3. 建议性纠正：自动纠正非标准名称、未来栈添加提示。

输出包含 signal、fixed_skills、issues 数组、hitl_required。

【示例输出】

(见前文，包括绿/黄/红灯三种情景)

概念性输出

JSON 对象，包含信号、修正后技能文本、问题列表及是否需要 HITL。

四、评估与分析

4.1 战略契合与 GenAI 合理性论证

业务对齐

公司目标：6 个月将技术岗招聘周期降至 48 天。V3.0 workflow 直接支撑：

- 前置校验减少返工
- 技能栈校验避免错配
- 迭代循环提升一次性通过率
- 自动生成缩短起草耗时

业务收益

- 效率：JD 起草耗时 ↓ 75%+，审阅周期 ↓ 70%+
- 质量：减少错误，吸引精准候选人
- 成本：降低人员时间成本

为什么选择 GenAI?

对比传统人工和基于规则的系统：GenAI 能理解非结构化需求、根据自然语言反馈动态修订、生成融合 EVP 的自然语言 JD、支持动态技术栈校验。在处理非结构化、动态变化、语言理解与生成任务上具有不可替代优势。

4.2 核心 KPI 指标

KPI 名称	基准值	目标值	统计口径 / 与战略关联
技术岗平均招聘周期	60 天	48 天	直接支撑公司战略 KPI
JD 一次性通过率	50%	90%	口径定义：未触发任何黄灯/红灯、且销售岗迭代次数 ≤ 1 次即进入统一审核平台的 JD 数量 $\div$ 总 JD 数量 $\times 100\%$ 。

4.3 风险、伦理与偏见分析

风险 1：需求一致性校验器误判

- 可能性：中
- 影响：中等至高（假阳性浪费人工，假阴性导致下游错误）
- 缓解策略：输出矛盾时推送给招聘专员确认（HITL），人工最终仲裁
- 残留风险：是（人工也可能疏忽，但概率低）

风险 2：EVP 融合度评分器不准确

- 可能性：中
- 影响：中等（增加无效迭代或低质量 JD 通过）
- 缓解策略：允许人工审核员手动覆盖评分；阈值可调（如 0.8 $\rightarrow$ 0.7）
- 残留风险：是（依赖审核员判断力）

风险 3：技能栈动态图谱数据源不完整或过时

- 可能性：中
- 影响：高（可能导致流程阻塞或生成错误技术栈要求）
- 缓解策略：图谱每日增量更新+每月人工审核；黄灯/红灯时 HITL 确认+紧急绕过机制；反馈闭环触发图谱更新
- 残留风险：是（完全避免成本极高，但通过 HITL 兜底可接受）

伦理

1. 责任归属

- 风险：AI 生成的 JD 可能含歧视性语言（如年龄、性别倾向）。
- 应对：Prompt 中明确禁止歧视性表述；设置 HITL 最终审核节点（统一人工审核平台），由招聘专员和业务经理共同确认。

2. 数据隐私

- 风险：邮件和电话笔记可能包含候选人非公开信息或敏感内容。
- 应对：输入数据仅用于当前 JD 生成，处理后不保留；系统设计层面支持脱敏处理（如自动移除姓名、联系方式）；访问权限仅限相关招聘人员。

偏见

1. 技能栈动态图谱校验器

- 偏见来源：图谱数据可能偏向当前主流技术栈，对新兴或小众技术产生误判（如将新技术判为“错配”）。
- 防范：保留“future_stack”字段并设置提示而非强制阻止；允许招聘专员在红灯时申请白名单并记录原因，供技术委员会审核。

2. 需求一致性校验器

- 偏见来源：若训练数据或 Prompt 偏向特定岗位描述风格，可能对非标准表述产生误判。
- 防范：使用清晰、中立的判断规则（如“完全互斥才记为矛盾”），并设置 HITL 确认，避免自动拒绝。

4.4 实施与变更管理

干系人分析与参与策略

干系人群体	关注点	参与策略
公司领导层	ROI 是否真降本增效；是否影响现有业务；资金支持	提供详细 ROI 测算 (见下文)；提议小范围试点用数据说话；展示分阶段投入计划 (AI 成本低)
公司员工 (招聘专员、业务经理)	学习成本高不高；是否会被 AI 取代	设计直观界面, HITL 操作简单；强调 AI 是助手；提供培训材料 (10 分钟视频+快速参考卡) 和试点期间专人支持

低流量岗位简化模式

对于连续 3 个月累计招聘量 ≤ 2 的岗位类别 (如法务、合规专员)，系统自动切换为 “轻量模式”：跳过技能栈校验和销售迭代循环，仅执行一致性校验后直接进入人工编辑平台，由招聘专员一键生成初稿，避免过度自动化。

试点成功验收标准 (量化门槛)

4 周技术岗试点成功的标志 (三项同时达成则批准全公司推广)：

- 1. 技术岗 JD 起草平均耗时 ≤ 45 分钟
- 2. JD 一次性通过率 $\geq 80\%$
- 3. 招聘经理满意度评分 $\geq 4.5/5$

HITL 的采纳与角色定位

HITL 设计提升用户参与度，强化 “人类是不可替代的一环” 的认知；通过主动决策和反馈建立信任，同时收集数据持续优化模型。

可扩展性与可维护性

维度	说明
可扩展性	增加新岗位类型只需扩展分类器和生成器；动态图谱自动更新；无状态 API 支持水平扩展
可维护性	各工具 Prompt 独立维护；阈值参数可配置；建议建立反馈日志定期复盘调优

4.5 ROI 详细计算

假设条件（通用）

- 招聘专员小时工资：80 元
- 业务经理小时工资：200 元
- AI API 调用成本：每个职位约 10 次，DeepSeek API 定价极低，约 2.5 元/职位
- 工具一次性设计与调优：2 人天 $\approx$ 5000 元，分摊 12 个月 $\approx$ 416 元/月（固定成本）

情景 1：月招聘技术岗位 20 个（基准情景）

成本侧（增量）

成本项	计算方式	金额（元/月）
新增 HITL 操作	$20 \text{ 个} \times 10 \text{ 分钟}/60 \times (80+200)/2 \text{ 元/小时}$	467
AI API 调用	$20 \text{ 个} \times 2.5 \text{ 元}$	50
工具分摊（固定）	$5000 \text{ 元} / 12 \text{ 个月}$	416
月度总增量成本		933

收益侧

收益项	计算方式	金额（元/月）
JD 起草时间节省	20 个 × 1.5 小时 × 80 元	2,400
JD 审阅时间节省	20 个 × 2.5 天 × 8 小时 × 200 元	80,000
减少返工和沟通成本 ¹	历史数据：每 JD 平均返工 2 次，每次沟通 1.5 小时，成本 420 元/JD；20 个岗位即 8,400 元，保守取	5,000
月度总收益		87,400
月度 ROI	$ROI = \frac{87,400 - 933}{933} \approx 92.7 \text{ 倍}$	

情景 2：月招聘技术岗位 10 个（保守情景）

成本侧（增量）

成本项	计算方式	金额（元/月）
新增 HITL 操作	10 个 × 10 分钟/60 × (80+200)/2 元/小时	233
AI API 调用	10 个 × 2.5 元	25
工具分摊（固定）	5000 元 / 12 个月	416
月度总增量成本		674

¹ 说明：以上计算中“减少返工和沟通成本”按保守取整（20 个岗位时取 5000 元，10 个岗位时取 2500 元），实际可根据企业历史数据调整。

收益侧

收益项	计算方式	金额（元/月）
JD 起草时间节省	10 个 × 1.5 小时 × 80 元	1,200
JD 审阅时间节省	10 个 × 2.5 天 × 8 小时 × 200 元	40,000
减少返工和沟通成本	历史数据：420 元/JD × 10 个 = 4,200 元，保守取	2,500
月度总收益		43,700

月度 ROI

$$ROI = \frac{43,700 - 674}{674} \approx 63.9 \text{ 倍}$$

结论

即使每月仅招聘 10 个技术岗位，ROI 仍高达 **63.9 倍**，商业合理性依然极强。固定工具分摊成本（416 元/月）在岗位数量减半时影响略有上升，但整体收益依然远超投入。这表明该方案在小规模招聘场景下同样具备显著的投资回报。

五、下一步行动

建议在 **技术岗招聘** 中开展为期 **4 周** 的**小范围试点**，验证 JD 生成效率与质量提升效果。试点同时收集以下数据用于全公司推广决策：

- 实际 JD 起草耗时与一次性通过率
- 招聘经理满意度评分
- HITL 节点平均处理时长

附件：

1. LLM 对话：
2. 核心概念性 AI 工具定义

工具 1：需求一致性校验器 v1.0（核心定义）

工具名称

需求一致性校验器 v1.0

目的

在招聘流程开始时，自动对比招聘经理的邮件需求与 HR 的电话补充笔记，识别两者之间是否存在**完全互斥的矛盾**（如一个要求“3-5 年经验”，另一个要求“5-8 年”），并输出结构化的一致性报告。若发现矛盾，触发 HITL 节点（招聘专员介入澄清），避免基于错误信息向下游传递。

概念性输入

- **招聘经理需求邮件文本**
假设关键字段：职位名称、职责描述片段、技能关键词、经验要求、项目优先级
- **HR 电话沟通补充笔记**
假设关键字段：职位名称、职责描述、技能要求、经验要求、项目紧急程度
- **公司与 EVP 文本**
（本工具暂不直接使用，为保持输入一致而保留）

核心 Prompt 设计（Level-3）

角色

你是一位严谨的 HR 需求一致性分析专家，擅长对比不同来源的招聘需求信息，识别其中是否存在****完全互斥的矛盾****。

背景

我将提供以下两份内容：

1. 招聘经理的需求邮件（M）
2. HR 的电话沟通补充笔记（P）

你的任务是逐字段对比这两份内容，判断是否存在互斥矛盾。****注意****：如果一方比另一方更详细、更具体，或者只是补充信息，不记为矛盾。只有当两者的说法****无法同时为真****（即完全互斥）时，才记为矛盾。

需要对比的字段

1. `job\_title` (职位名称)

- 互斥示例：“数据科学家” vs “数据分析师”
- 不矛盾示例：“数据科学家” vs “资深数据科学家” (后者是细化)

2. `experience\_years` (经验年限)

- 互斥示例：“3-5 年” vs “5-8 年” (区间完全不重叠)
- 不矛盾示例：“3-5 年” vs “3-6 年” (区间有重叠)

3. `core\_responsibilities` (核心职责描述)

- 互斥示例：“负责从 0 到 1 搭建平台” vs “只负责日常维护现有系统”
- 不矛盾示例：“负责从 0 到 1 搭建” vs “负责搭建并参与后续迭代”

4. `qualifications` (技能要求)

- 互斥示例：“必须精通 Python” vs “禁止使用 Python”
- 不矛盾示例：“精通 Python” vs “精通 Python 和 Java” (后者是补充)

5. `project\_priority` (项目优先级/紧急程度)

- 互斥示例：“紧急，需 48 天内完成” vs “常规，无明确时限”

任务步骤

1. 分别从邮件和电话笔记中提取上述 5 个字段的信息。
2. 对于每个字段，判断两个来源的信息是否存在完全互斥的矛盾。
3. 如果存在矛盾，将该字段记入 `conflicts` 数组，每条矛盾包含：
 - `field`：字段名称 (字符串)
 - `description`：用一句自然语言描述矛盾的具体内容 (自由文本，不强求统一模板)
4. 如果没有任何字段存在矛盾，则 `is\_consistent` 为 `true`，`conflicts` 为空数组。
5. 如果至少有一个字段存在矛盾，则 `is\_consistent` 为 `false`。

输出格式

严格合法的 JSON，不包含额外解释。

```
{
  "is_consistent": true | false,
  "conflicts": [
    { "field": "字段名", "description": "矛盾描述文字" }
  ]
}
```

【示例输出】

情景 1：无矛盾

```
{  
  "is_consistent": true,  
  "conflicts": []  
}
```

情景 2：存在矛盾

```
{  
  "is_consistent": false,  
  "conflicts": [  
    {  
      "field": "experience_years",  
      "description": "邮件要求经验 3-5 年，电话笔记要求 5-8 年，两者互斥"  
    }  
  ]  
}
```

概念性输出

一个 JSON 对象，包含 is_consistent（布尔值）和 conflicts（数组，每个元素含 field 和 description）。

工具 2：岗位分类与定级器 v1.0（核心定义）

工具名称

岗位分类与定级器 v1.0

目的

根据邮件、电话笔记以及一致性校验后的信息，自动判断该岗位的**类别**（技术岗/销售岗/通用岗）和**级别**（初级/中高级/管理层），驱动后续分支路径选择和 JD 生成器的内部适配。

概念性输入

- 邮件正文（原始需求）
- 电话补充记录（原始笔记）
- 一致性校验器输出的结构化 JSON（如果有矛盾已解决，使用修正后的信息）

假设关键字段：职位名称、职责描述片段、经验年限、关键词（如“带领团队”、“独立负责”等）。

核心 Prompt 设计（Level-3）

角色

你是一位 HR 岗位分类与定级专家。

背景

我将提供招聘经理的邮件、HR 的电话笔记, 以及经过一致性校验后的信息(如果有矛盾已处理)。
你需要判断该岗位的**类别**和**级别**。

类别判断规则

- **技术岗**：职位名称或职责中包含开发、工程师、数据、算法、架构、测试、运维等关键词。
- **销售岗**：职位名称或职责中包含销售、客户、市场、商务、增长等关键词。
- **通用岗**：不属于上述两类的岗位（如 HR、行政、财务、法务等）。

级别判断规则（优先级：更高层级优先）

级别	经验年限	关键词示例	
-----	-----	-----	-----
初级	0-2 年	协助、辅助、在指导下、基础	
中高级	3-5 年	独立负责、设计、优化、从 0 到 1、核心模块	
管理层	6 年以上	带领团队、管理、负责人、架构师、领导	

- 如果输入同时满足多个级别，优先选择**更高层级**（管理层 > 中高级 > 初级）。
- 如果没有明确信息，默认输出 **中高级**。

任务

综合邮件和笔记中的信息，输出一个 JSON 对象，包含：

- `job\_category`：字符串，值为“技术岗”/“销售岗”/“通用岗”
- `job\_level`：字符串，值为“初级”/“中高级”/“管理层”

输出格式

```
{"job_category": "技术岗", "job_level": "中高级"}
```

【示例输出】

```
{"job_category": "技术岗", "job_level": "中高级"}
```

概念性输出

一个包含 job_category 和 job_level 字段的 JSON 对象。

工具 3：EVP 融合度评分器 v1.0

工具名称

EVP 融合度评分器 v1.0

目的

评估销售岗 JD 草稿与公司 EVP（雇主价值主张）文本的匹配程度，输出 0-1 之间

的融合度分数和简短评语，驱动质量迭代循环。当分数 ≥ 0.8 时视为“良好”，进入人工审核节点；当分数低于 0.8 时触发 AI 自动微调，避免低质量 JD 流入下游流程。

概念性输入

- **销售岗 JD 草稿**：完整的四栏 JD 草稿（岗位亮点/公司文化 EVP/工作职责/任职要求），Markdown 格式
- **公司文化与 EVP 文本**：公司核心文化特征与 EVP 要素清单
- **职位名称**：当前岗位的名称（字符串）

核心 Prompt 设计 (Level-3)

角色

你是一位严谨的雇主品牌与招聘质量评估专家，擅长从候选人视角评估 JD 文案与公司 EVP 的一致性，识别融合度不足或表达失真的问题。

背景

我将提供以下两份内容：

1. 销售岗 JD 草稿 (JD)
2. 公司文化与 EVP 文本 (EVP)

你的任务是逐项评估 JD 草稿对 EVP 各要素的覆盖程度和融合质量，判断其是否具备市场竞争力。

****注意****：EVP 融合不是简单照搬 EVP 文本中的词汇，而是要将这些价值主张自然、有说服力地转化为面向销售候选人的岗位亮点和企业文化描述。

评分维度与标准

请从以下 5 个维度进行评估（每个维度 0-1 分，总分取平均）：

1. ****职业发展路径**** (Career Development)：JD 是否清晰传达了该岗位的成长空间、晋升通道、培训资源等？

- 1 分：详细且具体地描述了晋升阶梯（如“从 SDR 到区域经理的最快路径”）
- 0.5 分：有提及发展机会，但表述笼统
- 0 分：完全未提及

2. ****收入竞争力**** (Earning Potential)：JD 是否真实、有吸引力地呈现了薪酬结构（提成机制、奖金、激励方案等）？

- 1 分：清晰说明提成比例、不设上限、优秀者收入范围等具体信息
- 0.5 分：笼统提及“有竞争力的薪酬”
- 0 分：仅写“面议”或完全未提及收入相关

3. ****资源与产品力** (Resources & Product Strength)**: JD 是否让候选人感受到公司产品的市场竞争力和销售支持资源?

- 1 分: 具体描述了产品优势、市场地位、销售工具支持
- 0.5 分: 有提及但不够具体
- 0 分: 完全未提及

4. ****文化与包容性** (Culture & Inclusion)**: JD 是否体现了公司的文化特质 (如“快速迭代、拥抱创新、结果导向”) 和多元包容的工作环境?

- 1 分: 用具体场景或语言风格让文化鲜活可感
- 0.5 分: 照搬了文化词汇但未展开
- 0 分: 完全未体现

5. ****工作与成长平衡** (Work & Growth Balance)**: JD 是否传达了公司在“拼搏”与“支持成长”之间的平衡理念?

- 1 分: 同时体现了高成长机会与持续学习的支持
- 0.5 分: 仅侧重一方面
- 0 分: 完全未体现

任务步骤

1. 阅读 EVP 文本, 提取核心要素作为评估基准。
2. 逐项对照 5 个维度, 评估 JD 草稿中的对应内容。
3. 给出每个维度的单项分数和总分 (总分 = 各维度分数之和 / 5)。
4. 生成一段简短评语 (不超过 80 字), 明确指出分数的主要贡献项和短板项。

评分标尺与决策规则

- score $\geq$ 0.8 $\rightarrow$ “良好”, 可进入人工审核节点
- score < 0.8 $\rightarrow$ “待优化”, 需触发 AI 自动微调

输出格式

严格合法的 JSON, 不包含额外解释。

```
{  
  "score": 0.85,  
  "feedback": "职业发展路径和收入竞争力描述较好, 但文化与包容性体现不足, 建议补充公司多元包容的工作环境描述"
```

}

概念性输出

一个 JSON 对象，包含 score（浮点数，精确到小数点后两位）和 feedback（字符串，不超过 80 字）。

工具 4：反馈解析与修订器 v1.0

工具名称

反馈解析与修订器 v1.0

目的

将人工审核员或招聘专员在 S-JD-Review 节点提交的自然语言反馈意见，解析为结构化的修订指令，并自动生成修订后的 JD 草稿。该工具支持多轮迭代（最大 3 次），在保持原稿整体框架和结构的前提下，仅对反馈中明确指向的部分进行精准修改，并使用加粗标记高亮变更内容，便于审核员快速定位差异，降低审阅负担。

概念性输入

- **当前 JD 草稿**：上一轮迭代生成的完整 JD 草稿（Markdown 格式）
- **人工反馈文本**：审核员的自然语言反馈意见，格式示例：“职责第 2 条‘完成销售任务’太模糊，请结合我们 SaaS 产品的客单价和平均成交周期，修改为：‘独立负责从商机发现到回款的全销售周期，入职半年内需累计完成 XX 万元合同额。’”
- **EVP 文本**：公司文化与 EVP 要素清单（用于修订时确保 EVP 一致性）
- **职位名称**：当前岗位的名称（字符串）
- **迭代轮次**：当前修订轮次（整数，1-3）

核心 Prompt 设计 (Level-3)

角色

你是一位精准的 JD 修订专家，擅长解析人工审核员的自然语言反馈，并精确执行修改指令，同时保持 JD 整体质量的稳定性。

背景

我将提供以下内容：

1. 当前版本 JD 草稿（current_jd）
2. 人工审核员的反馈文本（feedback）
3. 公司 EVP 文本（evp_text，作为修订时的参考基准）
4. 当前修订轮次（iteration_round，1-3）

你的任务是将反馈文本解析为具体的修订动作，并在 JD 草稿中执行修改。****核心原则****：

- 只修改反馈中明确指出的部分，不要擅自调整其他内容
- 如果反馈同时指出多个问题，依次全部处理

- 保持 JD 原有的四栏结构（岗位亮点/公司文化 EVP/工作职责/任职要求）
- 所有修改处使用****加粗标记****（Markdown 加粗格式）高亮
- 如果某个反馈指令存在歧义，优先选择最小改动方案
- 如果指令要求删除某部分内容，直接删除并标记删除位置为“**【已删除】**”

反馈解析规则

1. ****定位指令****：识别反馈中涉及的 JD 章节和条款编号（如“职责第 2 条”“任职要求第 3 条”“亮点部分”）
2. ****动作识别****：判断操作类型——修改、新增、删除
3. ****内容提取****：提取需要写入的新内容

常见反馈类型与处理示例

- ****修改类型****：“职责第 2 条‘XXX’修改为‘YYY’” → 定位到对应条款，替换内容，加粗修改后的整句
- ****新增类型****：“任职要求中补充 Python 技能要求” → 在对应章节末尾追加，加粗新增内容
- ****删除类型****：“删除任职要求中关于 SQL Server 的表述” → 定位并删除，标记为“**【已删除】**”
- ****模糊指令****：“职责部分太泛了，需要更具体” → 结合 EVP 文本和行业标准，适当充实内容，加粗所有新增或修改的部分

输出格式

直接输出完整的修订后 JD 草稿（Markdown 格式），使用****加粗标记****高亮所有修改部分。输出中不包含 JSON 结构或额外解释。

****【输出示例片段】****

工作职责

1. ****独立负责从商机发现到回款的全销售周期管理****
2. 维护和拓展重点客户关系
3. ****入职半年内累计完成 200 万元合同额****

概念性输出

完整的 JD 草稿（Markdown 格式），所有修改部分均使用****加粗标记****高亮。

工具 5：熔断机制控制器 v1.0

工具名称

熔断机制控制器 v1.0

目的

在销售岗质量迭代循环中，当 AI 修订轮次达到最大尝试次数 (≥ 3 次) 仍无法通过人工审核时，自动触发熔断保护机制：强制将当前版本 JD 草稿提交至分支汇合点，避免无限循环阻塞招聘流程；同时向上级（HRBP 和招聘总监）发送升级提醒消息，附带未通过原因摘要和历史反馈记录，便于快速人工介入诊断问题根源（如需求本身存在底层矛盾）。

概念性输入

- 当前迭代次数：已执行的修订轮次（整数）
- 最后一次修订的 JD 草稿：第 N 轮修订后的完整 JD（Markdown 格式）
- 未通过的反馈记录：前 N 轮迭代中所有人工审核不通过的反馈文本数组
- 职位名称：当前岗位的名称（字符串）
- 评分历史：历次 EVP 评分记录（可选，用于更完整的诊断）

核心 Prompt 设计 (Level-3)

角色

你是一个工作流熔断控制器，负责监控迭代循环状态并执行异常出口逻辑。

背景

我正在监控销售岗 JD 质量迭代循环。已知：

- 最大迭代次数上限：3 次
- 当前已完成修订次数：iteration_round
- 人工审核结果：未通过

我需要判断是否达到熔断条件，若达到则：

- ① 生成“强制提交”信号
- ② 生成升级提醒消息，发送给 HRBP 和招聘总监

熔断逻辑规则

1. **条件判断**：IF iteration_round $\geq$ 3 AND 人工审核结果为“不通过” → 触发熔断
2. **强制提交**：将最后一次修订的 JD 草稿直接发送至分支汇合点，附加“熔断强制提交”标记
3. **升级提醒生成**：
 - 从未通过的反馈记录中提取所有反馈内容
 - 诊断问题类型（参考分类规则）
 - 输出升级提醒消息

问题类型诊断规则

根据反馈记录中的关键词，自动归类未通过的主要原因：

问题类型	关键词示例	建议措施
EVP 融合度不足	“EVP”“文化匹配”“品牌传达”	重新审查岗位与 EVP 的适配性
职责与业务需求偏差	“职责偏离”“与业务不符”“未反映实际需求”	邀请业务经理重新明确需求
任职要求过紧/过松	“要求太高”“门槛过低”“筛选不准确”	校准岗位级别与市场基准
结构或表述问题	“表述模糊”“结构混乱”“语言不够专业”	检查 EVP 评分器阈值设定是否合理

输出格式

系统不输出 JSON，而是执行两个动作并返回结果：

1. **强制提交信号**：内部标记 + 返回终稿内容
2. **升级提醒消息**（Markdown 格式）

【升级提醒消息模板】

🔔 【熔断提醒】

> **岗位名称**：{job_title}

> **熔断触发**：已完成 {iteration_round} 次迭代，仍未能通过人工审核。

> **历史反馈摘要**：

- > - 第 1 轮反馈：{feedback_1}
- > - 第 2 轮反馈：{feedback_2}
- > - 第 3 轮反馈：{feedback_3}

> **诊断结论**：根据反馈记录判断，主要问题可能属于“{问题类型}”。

> **建议措施**：{根据问题类型给出的建议}

> **已强制提交版本**：第 {iteration_round} 版 JD 草稿，请 HRBP 和招聘总监关注并人工介入协调。

【示例】

🔔 【熔断提醒】

> ****岗位名称****: 大客户销售经理

> ****熔断触发****: 已完成 3 次迭代, 仍未能通过人工审核。

> ****历史反馈摘要****:

>- 第 1 轮反馈: EVP 融合度不足, 职责描述与业务需求有偏差

>- 第 2 轮反馈: 任职要求中的提成机制描述不够吸引人

>- 第 3 轮反馈: 岗位亮点部分仍然过于泛泛

> ****诊断结论****: 根据反馈记录判断, 主要问题可能属于“职责与业务需求偏差”和“EVP 融合度不足”复合类型。

> ****建议措施****: 建议业务经理与 HRBP 共同重新梳理该岗位的核心定位, 确认 EVP 中的“收入竞争力”要素是否在本岗位中适用。

> ****已强制提交版本****: 第 3 版 JD 草稿, 请 HRBP 和招聘总监关注并人工介入协调。

概念性输出

- **强制提交信号**: 内部状态标记, 用于流程引擎判断, 将当前 JD 草稿推送至分支汇合点
- **升级提醒消息**: 人类可读的 Markdown 格式文本, 发送至预设的 HRBP 和招聘总监接收渠道

工具 6: 技能栈动态图谱校验器 v1.0

工具名称

技能栈动态图谱校验器 v1.0

目的

在技术岗 JD 生成之前, 自动将岗位分类与定级器输出的岗位需求 (JD 中的技能要求) 与公司内部技术资产库生成的动态技术栈图谱进行对比校验, 识别三类问题: 技能过时 (要求已淘汰技术)、技能错配 (偏离当前核心栈)、强制缺失 (缺少 core_stack 必备技术)。输出校验结果 (绿灯/黄灯/红灯) 并触发相应动作: 绿灯直接进入 JD 生成器; 黄灯自动修正或高亮后轻量级 HITL 确认; 红灯阻止流程并强制招聘专员修改, 避免生成基于错误技术栈的 JD, 降低面试筛选成本和 onboarding 技术债。

概念性输入

- **动态技术栈图谱 (JSON)**: 从公司技术资产库 (GitLab/GitHub、Confluence/Jira、HRIS 技能矩阵) 实时采集并聚合生成, 包含三个核心字段:
 - core_stack: 团队当前必须掌握的核心技术列表 (如 ["Python 3.10+", "PyTorch 2.0", "ClickHouse", "Kubernetes"])
 - legacy_stack: 仍在维护但不再扩展的技术列表 (如 ["Hadoop Hive", "Java 8"]), 每个技术附带推荐替代项
 - future_stack: 未来 6 个月计划引入的技术列表 (可选, 用于前瞻性

提示)

- **岗位技能需求文本**: 从岗位分类与定级器输出的分类信息 + 原始的邮件/笔记中提取的技能描述 (例如: “要求熟悉 Java 8、Hadoop、Spring Boot”)
- **职位名称**: 当前技术岗位名称 (字符串)

核心 Prompt 设计 (Level-3)

角色

你是一位资深的技术栈合规专家与技术招聘审核官, 精通企业技术资产治理, 能够快速识别岗位技能要求与公司真实技术栈之间的冲突、过时和缺失问题。

背景

我将提供以下两份内容:

1. 动态技术栈图谱 (tech_stack): 包含 core_stack (核心技术必备)、legacy_stack (淘汰中技术+替代项)、future_stack (未来技术)
2. 岗位技能需求文本 (job_skills): 招聘专员填写的技能要求片段

你的任务是对比分析, 判断技能需求是否可安全用于 JD 生成, 输出校验结果 (绿灯/黄灯/红灯) 及具体诊断信息。

校验规则 (严格按优先级执行)

第一阶段: 强制缺失检测 (最高优先级)

检查 job_skills 中是否**完整覆盖** tech_stack.core_stack 中的每一项核心技术 (忽略版本号微小差异, 如 Python 3.10 视为覆盖 3.9+)。

- **若缺失任意一项 core_stack 技术** → 直接判定为 ****红灯****
- 输出信息中必须列出缺失项, 并提示“必须包含, 不允许跳过”

第二阶段: 冲突与过时检测

仅当强制缺失检测通过后 (即 core_stack 已全覆盖), 执行以下检查:

1. ****legacy_stack 禁止性检测****: 若 job_skills 中出现 legacy_stack 中的任一技术, 并且描述词包含“必须精通/熟悉/掌握”等强制性词汇 → 判定为 ****黄灯****

- 输出信息中列出该技术及推荐替代项 (来自图谱)
- 动作: 系统自动将“必须”改为“了解或有迁移经验者优先”, 并高亮提示招聘专员确认

2. ****严重错配检测 (红灯条件)****: 若 job_skills 中出现与 core_stack 完全互斥的技术组合 (例如 core_stack 全为 Python + PyTorch, 但要求“Java”且非补充角色) → 判定为 ****红灯****

- 互斥判断依据: 该技术不在 core_stack、不在 future_stack、且不是通用辅助技术 (如 Git、Docker 视为通用, 不互斥)

- 输出信息：“严重冲突，请修改为 [推荐 core_stack 中的同类技术]”

第三阶段：建议性纠正

仅当未触发红灯时，执行以下辅助纠正：

- ****相似度推荐****：若技能名称非标准写法（如“pytorch1.9”）→ 自动纠正为图谱中的官方版本（如“PyTorch 1.9”或团队实际版本）
- ****未来栈提示****：若技能出现在 future_stack 中且当前团队尚不稳定 → 添加绿色提示“计划引入中，当前该岗位建议先关注 core_stack”

输出格式与信号定义

| 信号 | 条件 | 后续动作 |

|-----|-----|-----|

| 绿灯 | core_stack 全覆盖，无 legacy 强制词，无严重错配 | 直接进入技术岗 JD 生成器 |

| 黄灯 | core_stack 全覆盖，但存在 legacy 强制词或可自动修正项 | 系统自动修正后，推送给招聘专员轻量级确认 |

| 红灯 | 缺失 core_stack 项 或 严重错配 | 阻止流程，强制招聘专员修改后再提交 |

输出 JSON 格式

严格合法的 JSON，不包含额外解释。

```
{
  "signal": "green" | "yellow" | "red",
  "fixed_skills": "经过自动纠正后的技能需求文本（绿灯/黄灯时必填，红灯时为原始文本或 null）",
  "issues": [
    {
      "type": "missing_core" | "legacy_forbidden" | "severe_mismatch" | "auto_corrected" |
      "future_hint",
      "original": "原技能表述",
      "suggestion": "建议修改内容或解释"
    }
  ],
  "hitl_required": true | false
}
```

注：signal 为“黄灯”时 hitl_required = true（需要招聘专员确认）；“红灯”时流程阻塞，也视为需要

人工介入 (true); “绿灯”时为 false。

示例输出

**** 【情景 1: 绿灯】 ****

```
{
  "signal": "green",
  "fixed_skills": "要求: Python 3.10+、PyTorch 2.0、ClickHouse、Kubernetes",
  "issues": [
    {
      "type": "auto_corrected",
      "original": "pytorch1.9",
      "suggestion": "已纠正为 PyTorch 2.0 (团队当前版本) "
    }
  ],
  "hitl_required": false
}
```

**** 【情景 2: 黄灯 (legacy 强制)] ****

```
{
  "signal": "yellow",
  "fixed_skills": "了解 Hadoop Hive 或有迁移经验者优先; 必须熟练使用 ClickHouse、Python 3.10+",
  "issues": [
    {
      "type": "legacy_forbidden",
      "original": "必须精通 Hadoop Hive",
      "suggestion": "该技术正在淘汰, 已自动改为'了解或有迁移经验者优先', 推荐替代项为 ClickHouse + DBT。请确认修改。"
    }
  ],
  "hitl_required": true
}
```

**** 【情景 3: 红灯 (缺失 core)] ****

```
{
  "signal": "red",
  "fixed_skills": null,
  "issues": [
```

```
{
  "type": "missing_core",
  "original": "岗位技能描述中未提及 Kubernetes",
  "suggestion": "core_stack 必须包含 Kubernetes, 请补充该技能要求, 不可跳过。"
}
],
"hitl_required": true
}
```

概念性输出

一个 JSON 对象, 包含:

- signal: 字符串, 值为 "green" / "yellow" / "red"
- fixed_skills: 字符串 (修正后的技能需求文本) 或 null
- issues: 数组, 每个元素包含 type、original、suggestion 字段
- hitl_required: 布尔值, 指示是否需要人工介入